



Controle de Iluminação com ESP32

Gabriel Marcelino da Silva, Professor: Andre Luis de oliveira

Faculdade de Computação e Informática
Universidade Presbiteriana Mackenzie (UPM) – São Paulo, SP – Brazil

10722757@mackenzista.com.br

Abstract. *The Internet of Things (IoT) enables physical objects to connect to the internet, allowing communication, automation, and remote control of intelligent systems. This project presents the development of an intelligent urban lighting system using the ESP32 microcontroller integrated with Wi-Fi communication and the MQTT protocol. The proposed solution uses a light sensor (LDR) to monitor ambient luminosity and automatically control a public lighting unit represented by a LED connected through a relay module. In addition, the system includes visual indicators for Wi-Fi and MQTT connection status, improving system monitoring and reliability. The ESP32 communicates with an MQTT broker through TCP/IP communication, enabling real-time connectivity between the embedded system and IoT infrastructure. The project demonstrates how low-cost hardware can be applied to smart city solutions focused on energy efficiency and sustainable urban infrastructure.*

Resumo. *A Internet das Coisas (IoT) permite que objetos físicos sejam conectados à internet, possibilitando comunicação, automação e controle remoto de sistemas inteligentes. Este projeto apresenta o desenvolvimento de um sistema inteligente de iluminação urbana utilizando o microcontrolador ESP32 integrado à comunicação Wi-Fi e ao protocolo MQTT. A solução proposta utiliza um sensor de luminosidade (LDR) para monitorar a iluminação do ambiente e controlar automaticamente um ponto de iluminação pública representado por um LED acionado por meio de um módulo relé. Além disso, o sistema possui LEDs indicadores responsáveis por informar o status das conexões Wi-Fi e MQTT, aumentando o monitoramento e a confiabilidade da solução. O ESP32 realiza comunicação com um broker MQTT utilizando comunicação TCP/IP, permitindo conectividade em tempo real entre o sistema embarcado e a infraestrutura IoT. O projeto demonstra como hardwares de baixo custo podem ser aplicados em soluções de cidades inteligentes voltadas à eficiência energética e à sustentabilidade urbana, relacionando-se diretamente ao ODS 11 – Cidades e Comunidades Sustentáveis.*

1. Introdução

O avanço das tecnologias digitais e da conectividade impulsionou o crescimento da Internet das Coisas (IoT), permitindo que dispositivos físicos sejam conectados à internet para realizar monitoramento, automação e troca de informações em tempo real. Atualmente, soluções baseadas em IoT vêm sendo aplicadas em diferentes áreas da infraestrutura urbana, principalmente em sistemas relacionados à eficiência energética e cidades inteligentes (WOKWI, 2026).

Nesse contexto, os sistemas de iluminação pública representam uma importante aplicação da IoT, pois estão diretamente ligados ao consumo de energia elétrica e à gestão urbana. O presente projeto propõe o desenvolvimento de um sistema inteligente de controle de iluminação urbana utilizando o microcontrolador ESP32, um sensor de luminosidade LDR e comunicação via protocolo MQTT. O sistema realiza o monitoramento da luminosidade do ambiente e aciona automaticamente um ponto de iluminação representado por um LED conectado a um módulo relé. Além disso, o projeto utiliza LEDs indicadores para demonstrar o status das conexões Wi-Fi e MQTT.

A comunicação do sistema ocorre por meio do protocolo MQTT, amplamente utilizado em aplicações IoT devido à sua eficiência e baixo consumo de recursos (KNOLLEARY, 2026). O ESP32 conecta-se a um broker MQTT via internet utilizando comunicação TCP/IP, permitindo integração entre o sistema embarcado e a infraestrutura IoT proposta.

Além da aplicação tecnológica, o projeto relaciona-se diretamente ao ODS 11 – Cidades e Comunidades Sustentáveis (BRASIL, 2026), que busca promover cidades mais eficientes e sustentáveis por meio do uso inteligente de recursos. Dessa forma, o sistema contribui para a automação da iluminação urbana, reduzindo desperdícios energéticos e incentivando o desenvolvimento de soluções voltadas às cidades inteligentes.

2. Materiais e Métodos

2.1. Descrição Geral do Sistema

O presente projeto propõe o desenvolvimento de um sistema inteligente de controle de iluminação urbana baseado em conceitos de Internet das Coisas (IoT). A solução simula o funcionamento de um ponto de iluminação pública conectado, realizando o monitoramento da luminosidade do ambiente e o acionamento automático da iluminação de acordo com as condições do local.

O sistema é composto por um microcontrolador ESP32 responsável pelo processamento de dados e comunicação com a internet, um sensor LDR utilizado para medir a luminosidade do ambiente e um módulo relé responsável pelo acionamento da iluminação representada por um LED. Além disso, o projeto utiliza LEDs indicadores para monitoramento visual do status das conexões Wi-Fi e MQTT.

A comunicação entre os dispositivos ocorre por meio do protocolo MQTT, amplamente utilizado em aplicações IoT devido à sua leveza e eficiência na troca de mensagens em tempo real. O ESP32 conecta-se a um broker MQTT público via internet utilizando comunicação TCP/IP, permitindo integração entre o sistema embarcado e a infraestrutura IoT. Para desenvolvimento e simulação do sistema foram utilizadas a plataforma Wokwi, ferramentas amplamente utilizadas em projetos embarcados e automação.

2.2 Plataforma de Prototipagem

A plataforma escolhida para o desenvolvimento do projeto é o ESP32, um microcontrolador de baixo custo amplamente utilizado em aplicações de Internet das Coisas (IoT). Segundo a documentação da plataforma Wokwi (2026), o ESP32 possui conectividade Wi-Fi integrada, permitindo comunicação direta com a internet sem a necessidade de módulos adicionais.

Além da conectividade, o ESP32 apresenta boa capacidade de processamento, múltiplas portas de entrada e saída (GPIO) e suporte a diferentes protocolos de comunicação, características que o tornam adequado para aplicações de automação e sistemas embarcados. No projeto proposto, o microcontrolador é responsável pelo processamento das informações recebidas pelo sensor LDR, pelo controle do módulo relé e pela comunicação com o broker MQTT via protocolo TCP/IP.

Essas características permitem a aplicação do ESP32 em soluções voltadas à infraestrutura urbana inteligente, como sistemas de iluminação pública conectada, possibilitando monitoramento, automação e troca de dados em tempo real entre dispositivos IoT.

2.3 Componentes Utilizados

Para a implementação do sistema, serão utilizados os seguintes componentes:

a) Microcontrolador (ESP32)

O microcontrolador ESP32 será o principal componente do sistema, sendo responsável pelo processamento das informações, controle dos dispositivos conectados e comunicação com a internet. Trata-se de uma plataforma amplamente utilizada em aplicações de Internet das Coisas devido à sua alta capacidade de processamento, baixo consumo energético e conectividade integrada (WOKWI, 2026).

O ESP32 possui suporte nativo às tecnologias Wi-Fi e Bluetooth, permitindo comunicação direta com redes e dispositivos sem a necessidade de módulos adicionais. Além disso, dispõe de múltiplas portas de entrada e saída (GPIO), possibilitando a integração com sensores, atuadores e módulos eletrônicos utilizados em sistemas embarcados e automação.

No projeto proposto, o ESP32 atua como unidade central de controle do sistema de iluminação urbana inteligente. O microcontrolador realiza a leitura dos dados provenientes do sensor LDR por meio de portas analógicas ADC, processa as informações de luminosidade do ambiente e executa o acionamento automático do módulo relé responsável pelo controle da iluminação pública simulada.

Além do processamento local, o ESP32 também realiza comunicação com a internet utilizando protocolos TCP/IP e MQTT. Para isso, são utilizadas as bibliotecas WiFi.h e PubSubClient.h, responsáveis respectivamente pela conexão do dispositivo à rede Wi-Fi e pela comunicação com o broker MQTT (KNOLLEARY, 2026). Essa estrutura permite

a integração do sistema à infraestrutura IoT proposta no projeto, possibilitando monitoramento e troca de informações em tempo real.

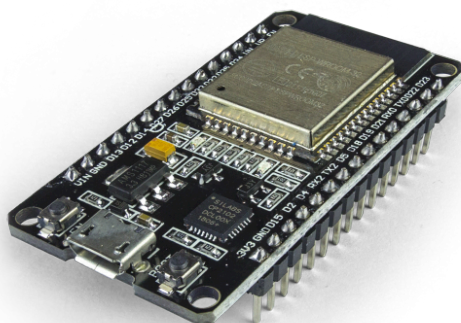


Figura 1 – Microcontrolador ESP32

Fonte: Disponível em: <https://www.robocore.net/wifi/esp32-wifi-bluetooth>

b) Sensor de Luminosidade (LDR)

O projeto utiliza um módulo sensor de luminosidade baseado em LDR (Light Dependent Resistor), responsável pelo monitoramento da intensidade de luz presente no ambiente. Esse tipo de sensor opera a partir da variação da resistência elétrica do LDR conforme a quantidade de luminosidade incidente: quanto maior a presença de luz, menor sua resistência; em ambientes escuros, a resistência aumenta (MAKERHERO, 2026).

O módulo utilizado possui circuito integrado com saídas analógica (AO) e digital (DO), permitindo integração simplificada com o ESP32. No projeto, é utilizada a saída analógica do sensor, conectada a uma porta ADC do microcontrolador, permitindo a leitura contínua dos níveis de luminosidade do ambiente (ESP32IO, 2026).

O ESP32 realiza a leitura dos dados enviados pelo sensor e processa essas informações para controlar automaticamente o sistema de iluminação urbana simulado. Quando o ambiente apresenta baixa luminosidade, o sistema aciona o módulo relé responsável pela iluminação pública representada pelo LED. Em ambientes claros, a iluminação é desligada automaticamente.

Essa funcionalidade simula o funcionamento de sistemas inteligentes de iluminação pública utilizados em cidades inteligentes, contribuindo para maior eficiência energética, automação urbana e redução do desperdício de energia elétrica.

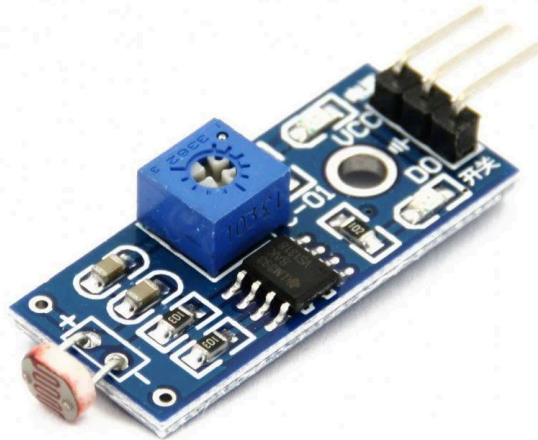


Figura 2 – Módulo Sensor de Luminosidade Luz LDR

Fonte: Disponível em:

https://www.eletrogate.com/modulo-sensor-de-luminosidade-ldr?utm_source=Site&utm_medium=GoogleMerchant&utm_campaign=GoogleMerchant&srsitid=AfmBOooFkRBO-Kv1PXKMd_EStkelJw6ATrlydK5S3ERnpTHPVB-npIBQ74M

c) Atuador (LED branco)

O LED branco será utilizado como principal atuador do sistema, representando um ponto de iluminação pública dentro do ambiente de prototipagem. Seu acionamento ocorre por meio do módulo relé controlado pelo ESP32, simulando o funcionamento de postes inteligentes presentes em sistemas modernos de iluminação urbana.

O controle desse LED ocorre automaticamente com base nos dados coletados pelo sensor de luminosidade. Quando o ambiente apresenta baixa luminosidade, o ESP32 aciona o relé responsável pela energização da iluminação pública simulada. Em ambientes claros, o sistema desliga automaticamente a iluminação, promovendo maior eficiência energética e redução do desperdício de energia elétrica.



Figura 3 – LED de alto brilho

Fonte: Eletrogate. Disponível em:<https://www.eletrogate.com/led-alto-brilho-5mm-verde>

d) Atuador (LED azul)

O LED azul será utilizado como indicador visual do status da conexão Wi-Fi do sistema. Durante o processo de conexão do ESP32 à rede, o LED permanecerá piscando, indicando tentativa de conexão com a internet. Após o estabelecimento da conexão, o LED permanecerá aceso continuamente, demonstrando que o sistema está conectado à infraestrutura de rede.

Essa funcionalidade permite monitoramento visual rápido do funcionamento do sistema IoT, auxiliando na identificação de falhas de conectividade durante a operação do protótipo.



Figura 4 – LED Azul

Fonte:Disponível em:

https://www.eletrogate.com/led-difuso-5mm-azul?utm_source=Site&utm_medium=GoogleMerchant&utm_campaign=GoogleMerchant&srsitid=AfmBOop1JTKSrhExtsf8SC4U-3-2JuYnFKne6F-bLh8oj9lVlWaW9mvggJM

e) Atuador (LED vermelho)

O LED vermelho será responsável por indicar o status da conexão MQTT entre o ESP32 e o broker MQTT utilizado pelo sistema. Quando a comunicação MQTT for estabelecida corretamente, o LED permanecerá aceso, indicando funcionamento adequado da comunicação IoT.

Caso ocorra perda de conexão com o broker MQTT, o LED permanecerá apagado, sinalizando falha na comunicação entre o dispositivo embarcado e a infraestrutura IoT. Essa funcionalidade permite acompanhar visualmente o estado da comunicação em tempo real, reforçando o conceito de monitoramento inteligente aplicado ao projeto.

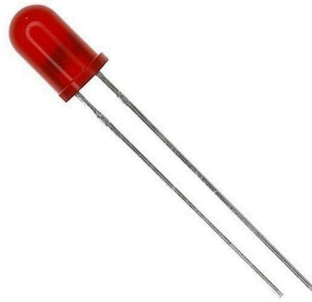


Figura 5 – LED vermelho

Fonte: Disponível em:

https://www.eletrogate.com/led-difuso-5mm-vermelho?utm_source=Site&utm_medium=GoogleMerchant&utm_campaign=GoogleMerchant&srsitid=AfmBOor4hqsOyAVtaljTkEt4ktsnZkZ0cuOhqrKykEmuBdpcHf4ZuhNCnzE

f) Resistores

Os resistores são componentes eletrônicos utilizados para limitar a passagem de corrente elétrica no circuito, garantindo proteção e funcionamento adequado dos dispositivos conectados. No projeto proposto, os resistores são utilizados principalmente nos LEDs indicadores do sistema, evitando a passagem de corrente excessiva que poderia causar danos aos componentes eletrônicos.

Cada LED utilizado no protótipo possui um resistor associado, responsável por controlar a intensidade da corrente elétrica enviada pelo ESP32 às portas GPIO. Essa configuração garante maior segurança e estabilidade no funcionamento do circuito eletrônico.

Diferentemente de montagens tradicionais com LDR discreto, o módulo sensor de luminosidade utilizado no projeto já possui circuito interno integrado, incluindo os resistores necessários para funcionamento do divisor de tensão e processamento dos sinais analógicos. Dessa forma, não foi necessária a utilização de resistores externos adicionais no circuito do sensor de luminosidade.

Os resistores utilizados no sistema contribuem diretamente para a confiabilidade do protótipo, garantindo proteção dos componentes e estabilidade elétrica durante o funcionamento da infraestrutura IoT simulada.



Figura 6 – Resistores eletrônicos

Fonte:Disponível em: <https://www.eletrogate.com/resistor-1m-1-4w-10-unidades>

g) Módulo relé

O módulo relé será utilizado como atuador responsável pelo acionamento da iluminação pública simulada no projeto. Esse componente funciona como um interruptor eletrônico controlado pelo ESP32, permitindo o acionamento de dispositivos elétricos a partir de sinais digitais enviados pelo microcontrolador.

No sistema proposto, o ESP32 envia sinais elétricos ao pino de controle do relé, permitindo ligar ou desligar o circuito responsável pela alimentação do LED que representa o ponto de iluminação urbana. Essa estrutura simula o funcionamento de sistemas reais de iluminação pública inteligente, nos quais centrais de controle realizam o acionamento remoto ou automático dos postes de iluminação.

O relé utilizado possui entradas de alimentação (VCC e GND), um pino de controle (IN) conectado ao ESP32 e terminais de chaveamento elétrico, como COM (comum) e NO (normalmente aberto). Quando o ESP32 identifica baixa luminosidade no ambiente por meio do sensor LDR, o módulo relé é acionado automaticamente, permitindo a energização da iluminação pública simulada.

A utilização do módulo relé no projeto demonstra a integração entre sensores, atuadores e sistemas embarcados em aplicações de automação urbana e infraestrutura inteligente, permitindo o controle automatizado da iluminação com maior eficiência energética e confiabilidade operacional.



Figura 7 – Módulo relé

Fonte: Disponível em:

https://www.eletrogate.com/modulo-rele-1-canal-5v?utm_source=Site&utm_medium=GoogleMerchant&utm_campaign=GoogleMerchant&srltid=AfmBOorIEoDpyuVJhCXBxhryMnBrvrQPD5TgOJao7bp4DCYL6hNc3tIH8E

f) Fios de Conexão

Os fios de conexão, também conhecidos como jumpers, são utilizados para realizar a interligação elétrica entre os componentes eletrônicos do circuito e o microcontrolador ESP32. Esses componentes permitem a transmissão de sinais elétricos, dados e alimentação entre os dispositivos presentes na prototipagem eletrônica.

No projeto, os jumpers foram utilizados para conectar o ESP32 ao módulo sensor de luminosidade LDR, ao módulo relé e aos LEDs indicadores responsáveis pelo monitoramento das conexões Wi-Fi e MQTT. Além disso, os fios também realizam as conexões de alimentação elétrica (5V e GND) necessárias para o funcionamento adequado dos módulos eletrônicos utilizados no sistema.

A utilização correta dos fios de conexão garante estabilidade elétrica, organização do circuito e comunicação eficiente entre sensores, atuadores e o sistema embarcado, sendo um elemento essencial para o funcionamento da infraestrutura IoT simulada no projeto.



Figura 8 – Fios de Conexão

Fonte: Disponível em: [40 Cabos Jumper 20cm Macho X Fêmea Para Arduino Protoboard | MercadoLivre](#)

2.4 Comunicação e Protocolo MQTT

A comunicação do sistema é realizada por meio do protocolo MQTT (Message Queuing Telemetry Transport), um protocolo leve baseado no modelo publish/subscribe, amplamente utilizado em aplicações de Internet das Coisas devido à sua eficiência e baixo consumo de recursos computacionais (KNOLLEARY, 2026).

No projeto, o ESP32 atua como cliente MQTT, conectando-se a um broker MQTT público por meio da internet utilizando comunicação TCP/IP. O broker utilizado é o HiveMQ, responsável por intermediar a troca de mensagens entre os dispositivos conectados ao sistema IoT.

A implementação da comunicação MQTT foi realizada utilizando a biblioteca PubSubClient.h, responsável pelo gerenciamento da conexão entre o ESP32 e o broker MQTT. Já a conectividade com a rede Wi-Fi foi implementada por meio da biblioteca WiFi.h, permitindo acesso do sistema à infraestrutura de rede (KNOLLEARY, 2026; WOKWI, 2026).

Durante o funcionamento do sistema, o ESP32 estabelece conexão com o broker MQTT e realiza o envio de informações relacionadas ao estado da iluminação urbana simulada. Além disso, o sistema utiliza um LED indicador para demonstrar visualmente o status da conexão MQTT em tempo real, permitindo monitoramento da comunicação IoT durante a execução do protótipo.

A utilização do protocolo MQTT permite comunicação eficiente, leve e escalável, sendo amplamente aplicada em soluções voltadas a cidades inteligentes, automação urbana e infraestrutura conectada.

2.5 Metodologia de Desenvolvimento

O desenvolvimento do projeto foi realizado em etapas, iniciando pela modelagem do circuito eletrônico em ambiente virtual de prototipagem utilizando a plataforma Wokwi (WOKWI, 2026). Nessa etapa, foram realizadas as conexões entre o microcontrolador ESP32, o módulo sensor de luminosidade LDR, o módulo relé e os LEDs indicadores responsáveis pelo monitoramento das conexões Wi-Fi e MQTT.

Posteriormente, foi realizada a programação do ESP32 utilizando a IDE Arduino e linguagem C/C++, empregando as bibliotecas WiFi.h e PubSubClient.h para implementação da comunicação Wi-Fi e integração com o protocolo MQTT. O sistema foi configurado para conectar-se automaticamente à rede Wi-Fi e estabelecer comunicação com um broker MQTT público via internet.

Após a implementação da conectividade, foi desenvolvida a lógica de automação do sistema, permitindo que o ESP32 realizasse a leitura contínua dos dados enviados pelo sensor de luminosidade por meio das portas ADC. Com base nos valores de luminosidade identificados, o microcontrolador executa automaticamente o acionamento do módulo relé responsável pelo controle da iluminação urbana simulada.

Além da automação da iluminação, foram implementados LEDs indicadores para monitoramento visual do funcionamento do sistema. O LED azul indica o status da conexão Wi-Fi, enquanto o LED vermelho demonstra o estado da comunicação MQTT entre o ESP32 e o broker MQTT.

Por fim, foram realizados testes de funcionamento no ambiente de simulação para validar a comunicação via internet, a leitura do sensor de luminosidade, o acionamento automático da iluminação e o funcionamento da infraestrutura IoT proposta no projeto.

2.6 Broker MQTT

Para a comunicação entre os dispositivos do sistema, foi utilizado o broker MQTT HiveMQ, uma plataforma amplamente empregada em aplicações de Internet das Coisas devido à sua estabilidade, facilidade de implementação e disponibilidade de servidores públicos para testes e prototipagem.

O broker MQTT é responsável por intermediar a troca de mensagens entre os dispositivos conectados à infraestrutura IoT, seguindo o modelo publish/subscribe. Nesse modelo, os dispositivos podem publicar informações em tópicos específicos e também receber mensagens relacionadas aos tópicos assinados.

No projeto desenvolvido, o ESP32 atua como cliente MQTT, realizando conexão com o broker HiveMQ via internet utilizando comunicação TCP/IP. Após estabelecer a conexão, o microcontrolador executa o envio de informações relacionadas ao funcionamento do sistema de iluminação urbana inteligente, permitindo integração entre o dispositivo embarcado e a infraestrutura de comunicação IoT proposta.

A comunicação MQTT foi implementada utilizando a biblioteca PubSubClient.h, responsável pelo gerenciamento da conexão entre o ESP32 e o broker MQTT. Durante o funcionamento do sistema, um LED indicador vermelho é utilizado para demonstrar visualmente o status da conexão MQTT, permitindo monitoramento em tempo real da comunicação estabelecida entre o sistema embarcado e o broker.

A escolha do HiveMQ ocorreu devido à sua compatibilidade com dispositivos embarcados, suporte ao protocolo MQTT e facilidade de integração com aplicações voltadas à automação e cidades inteligentes.

2.7 Ambiente de Desenvolvimento (IDE)

Para o desenvolvimento do software embarcado no ESP32, foi utilizada a Arduino IDE, uma plataforma amplamente empregada no desenvolvimento de projetos com microcontroladores e sistemas embarcados. A ferramenta oferece uma interface simples, suporte à linguagem C/C++ e integração com diferentes bibliotecas voltadas à automação e aplicações de Internet das Coisas.

A Arduino IDE foi utilizada para implementação do código responsável pela leitura do sensor de luminosidade, controle do módulo relé, gerenciamento dos LEDs indicadores e comunicação do ESP32 com a internet por meio da rede Wi-Fi.

Para implementação da conectividade do sistema, foram utilizadas as bibliotecas WiFi.h e PubSubClient.h. A biblioteca WiFi.h é responsável pela conexão do ESP32 à rede Wi-Fi, enquanto a biblioteca PubSubClient.h realiza o gerenciamento da comunicação MQTT entre o microcontrolador e o broker HiveMQ.

A utilização da Arduino IDE permitiu a integração eficiente entre hardware e software, facilitando o desenvolvimento da lógica de automação, comunicação IoT e monitoramento em tempo real do sistema de iluminação urbana inteligente proposto no projeto.

2.8 Funcionamento do Circuito e Lógica do Sistema

O funcionamento do sistema baseia-se na integração entre o módulo sensor de luminosidade LDR, o microcontrolador ESP32, o módulo relé e a comunicação via protocolo MQTT. O circuito foi desenvolvido na plataforma Wokwi utilizando conexões entre sensores, atuadores e módulos eletrônicos responsáveis pelo funcionamento da infraestrutura IoT simulada (WOKWI, 2026).

O módulo sensor de luminosidade utilizado no projeto possui saídas analógica (AO) e digital (DO), sendo utilizada a saída analógica conectada ao pino VP/GPIO36 do ESP32. Esse pino possui suporte ao conversor analógico-digital (ADC), permitindo que o microcontrolador realize a leitura contínua dos níveis de luminosidade do ambiente (ESP32IO, 2026).

O funcionamento do sensor ocorre por meio da variação da resistência do LDR conforme a intensidade luminosa do ambiente. Em ambientes iluminados, a resistência do sensor diminui, gerando valores analógicos diferentes dos observados em ambientes escuros. O ESP32 interpreta esses valores utilizando o ADC e executa a lógica de automação programada no sistema.

A lógica de funcionamento ocorre da seguinte forma:

- Em ambientes iluminados:
 - O sensor identifica alta luminosidade;
 - O ESP32 interpreta os valores analógicos acima do limite configurado;
 - O módulo relé permanece desligado;
 - O LED que representa a iluminação pública permanece apagado.
- Em ambientes escuros:
 - O sensor identifica baixa luminosidade;
 - O ESP32 interpreta os valores analógicos abaixo do limite estabelecido;

- O módulo relé é acionado automaticamente;
- O LED de iluminação pública é ligado.

Além da automação baseada no sensor, o sistema também incorpora comunicação via protocolo MQTT. O ESP32 conecta-se ao broker HiveMQ utilizando comunicação TCP/IP e mantém comunicação contínua com a infraestrutura IoT por meio da biblioteca PubSubClient.h.

O projeto também utiliza LEDs indicadores responsáveis pelo monitoramento visual do funcionamento do sistema. O LED azul indica o status da conexão Wi-Fi do ESP32, permanecendo piscando durante a tentativa de conexão e aceso após conexão estabelecida. Já o LED vermelho indica o status da comunicação MQTT, permanecendo aceso quando a conexão com o broker MQTT é realizada corretamente.

Dessa forma, o sistema combina automação baseada em sensores, comunicação via internet e monitoramento em tempo real, simulando o funcionamento de uma infraestrutura de iluminação urbana inteligente alinhada aos conceitos de cidades inteligentes, automação urbana e eficiência energética.

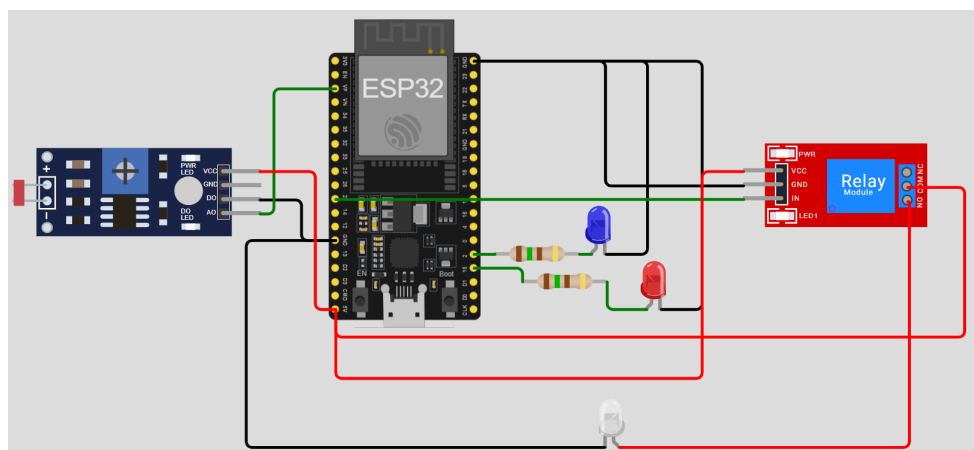


Figura 9 – modelo de montagem

2.9 Trecho de Implementação no Arduino IDE

```
#include <WiFi.h>

#include <PubSubClient.h>

// ===== WIFI =====

const char* ssid = "Wokwi-GUEST";
```

```

const char* password = "";

// ===== MQTT =====

const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;

PubSubClient client(espClient);

// ===== LEDs =====

const int LED_WIFI = 18;

const int LED_MQTT = 19;

// ===== SENSOR =====

#define LIGHT_SENSOR_PIN 36

#define RELAY_PIN 27

#define ANALOG_THRESHOLD 500

// ===== WIFI =====

void setup_wifi() {

    Serial.println();

    Serial.print("Connecting WiFi");

    WiFi.mode(WIFI_STA);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {

        delay(500);

        Serial.print(".");

        digitalWrite(LED_WIFI, !digitalRead(LED_WIFI));

    }

    Serial.println("\nWiFi connected");

```

```

    digitalWrite(LED_WIFI, HIGH);

}

// ===== MQTT =====

void reconnect() {

    while (!client.connected()) {

        Serial.print("Connecting MQTT...");

        String clientId = "ESP32-";

        clientId += String(random(0xffff), HEX);

        if (client.connect(clientId.c_str())) {

            Serial.println(" connected");

            client.subscribe("ESP_TESTE");

            digitalWrite(LED_MQTT, HIGH);

        } else {

            Serial.print(" failed rc=");

            Serial.println(client.state());

            digitalWrite(LED_MQTT, LOW);

            delay(5000);

        }

    }

}

// ===== SENSOR DE LUZ =====

void sensorLuz() {

    // INICIA CONTAGEM DE TEMPO

    unsigned long inicioSensor = millis();

```

```
int analogValue = analogRead(LIGHT_SENSOR_PIN);

Serial.print("Luminosidade: ");

Serial.println(analogValue);

// AMBIENTE ESCURO

if (analogValue > ANALOG_THRESHOLD) {

    digitalWrite(RELAY_PIN, HIGH);

    Serial.println("RELE LIGADO");

    // TEMPO ATUADOR

    unsigned long tempoAtuador = millis() - inicioSensor;

    Serial.print("Tempo resposta atuador: ");

    Serial.print(tempoAtuador);

    Serial.println(" ms");

    // ENVIO MQTT

    client.publish("cidade/luz/status", "LUZ LIGADA");

    // TEMPO MQTT

    unsigned long tempoMQTT = millis() - inicioSensor;

    Serial.print("Tempo envio MQTT: ");

    Serial.print(tempoMQTT);

    Serial.println(" ms");

} else {

    digitalWrite(RELAY_PIN, LOW);

    Serial.println("RELE DESLIGADO");

    client.publish("cidade/luz/status", "LUZ DESLIGADA");
```



```

    }

}

// ===== SETUP =====

void setup() {

    Serial.begin(115200);

    // LEDs

    pinMode(LED_WIFI, OUTPUT);

    pinMode(LED_MQTT, OUTPUT);

    // Relé

    pinMode(RELAY_PIN, OUTPUT);

    digitalWrite(LED_WIFI, LOW);

    digitalWrite(LED_MQTT, LOW);

    digitalWrite(RELAY_PIN, LOW);

    setup_wifi();

    client.setServer(mqtt_server, 1883);

}

// ===== LOOP =====

void loop() {

    if (!client.connected()) {

        reconnect();

    }

    client.loop();

```

```

sensorLuz();

delay(1000);

}

```

3. Resultados

3.1 Dificuldade

3.2 Funcionalidade

- ESP32 tenta se conectar automaticamente à internet. Enquanto essa conexão está sendo realizada, o LED azul fica piscando, indicando tentativa de conexão. Quando o Wi-Fi é conectado com sucesso, o LED azul permanece aceso.

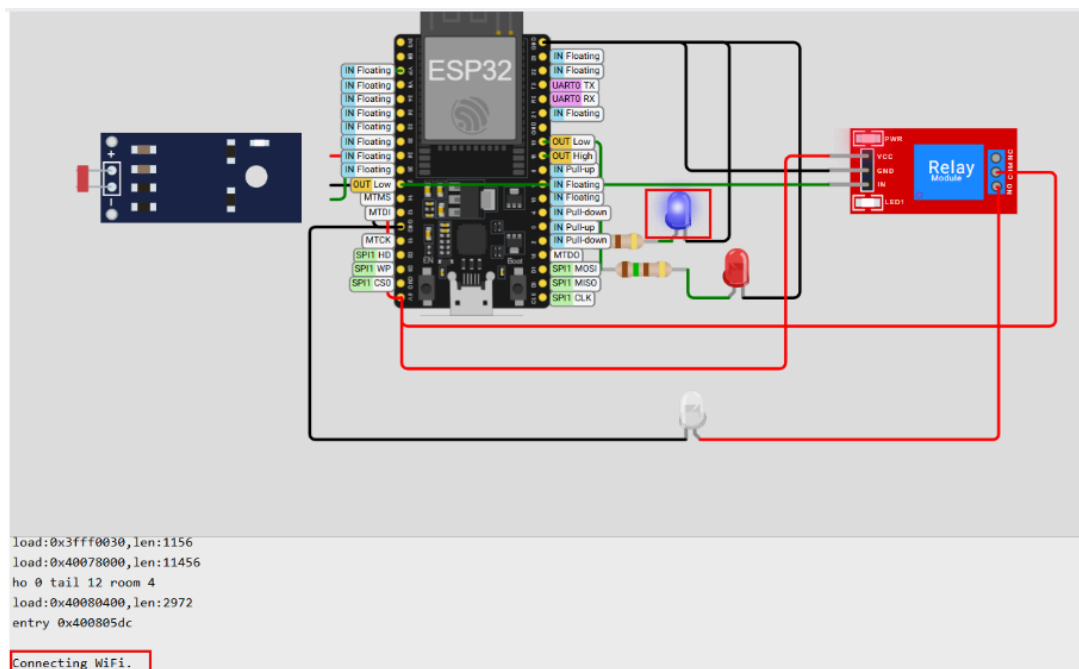


Figura 10 – LED azul piscando na tentativa de se conectar com Wifi

- O ESP32 inicia a comunicação MQTT utilizando a biblioteca PubSubClient.h. Se conecta ao broker MQTT HiveMQ, que funciona como um servidor responsável por intermediar a comunicação entre os dispositivos. Quando a conexão MQTT é estabelecida corretamente, o LED vermelho acende, mostrando que a comunicação com o broker está funcionando.

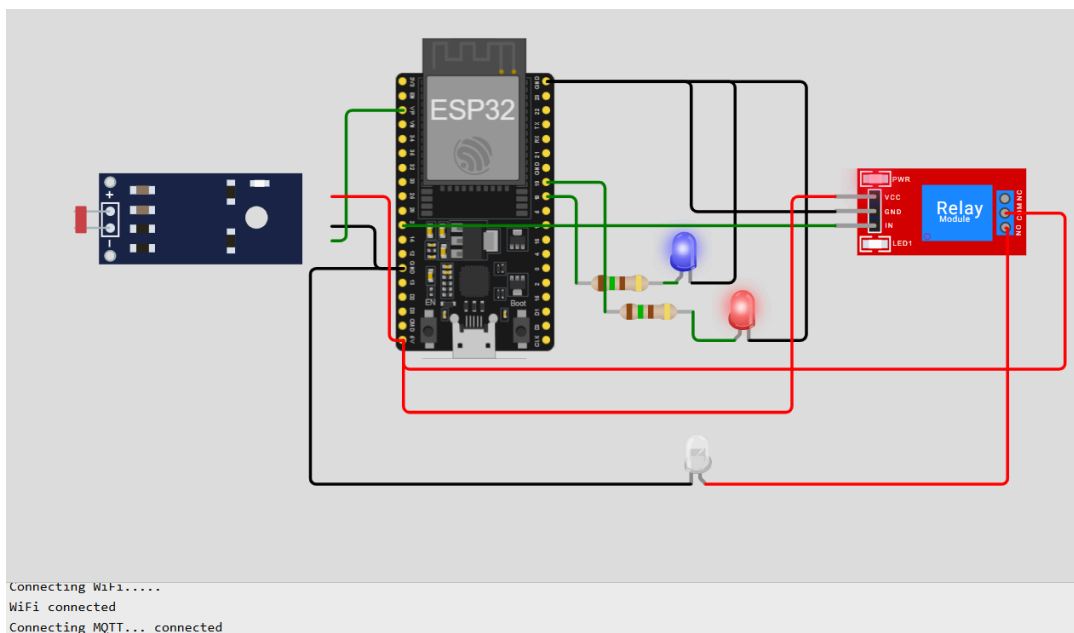


Figura 11 – LED vermelho aceso após conexão com MQTT

- Após isso, o ESP32 começa a monitorar continuamente os dados do sensor de luminosidade LDR. Quando o ambiente fica escuro, o sensor envia os valores para o ESP32, que processa essas informações e aciona automaticamente o relé responsável pela iluminação pública simulada.

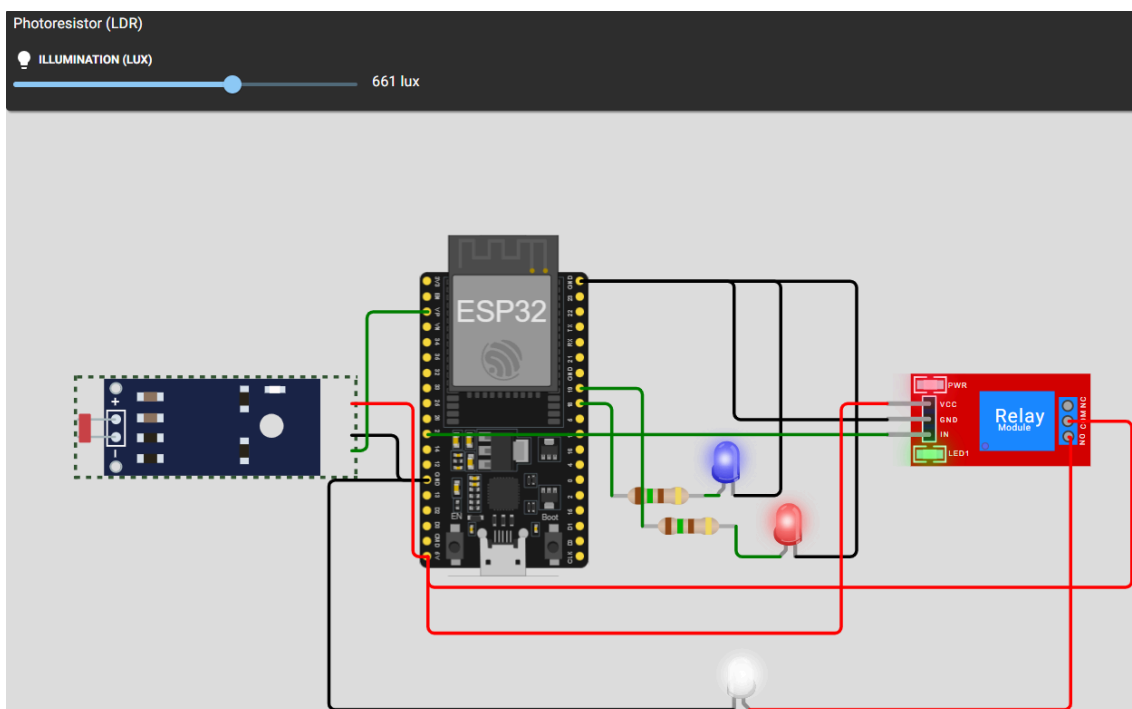


Figura 12 – LED branco aceso devido baixa luminosidade

- Após isso, o ESP32 começa a monitorar continuamente os dados do sensor de luminosidade LDR. Quando o ambiente fica claro, o sensor envia os valores para o ESP32, que processa essas informações e desliga automaticamente o relé responsável pela iluminação pública simulada.

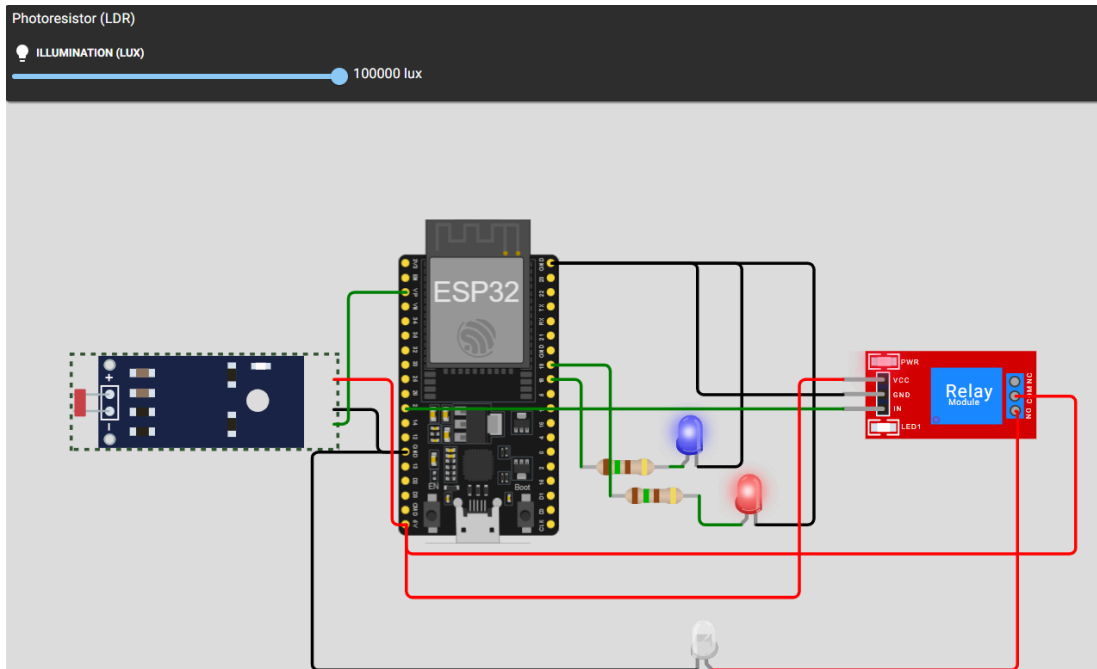


Figura 13 – LED branco apagado devido alta luminosidade

3.3 Análise de Desempenho

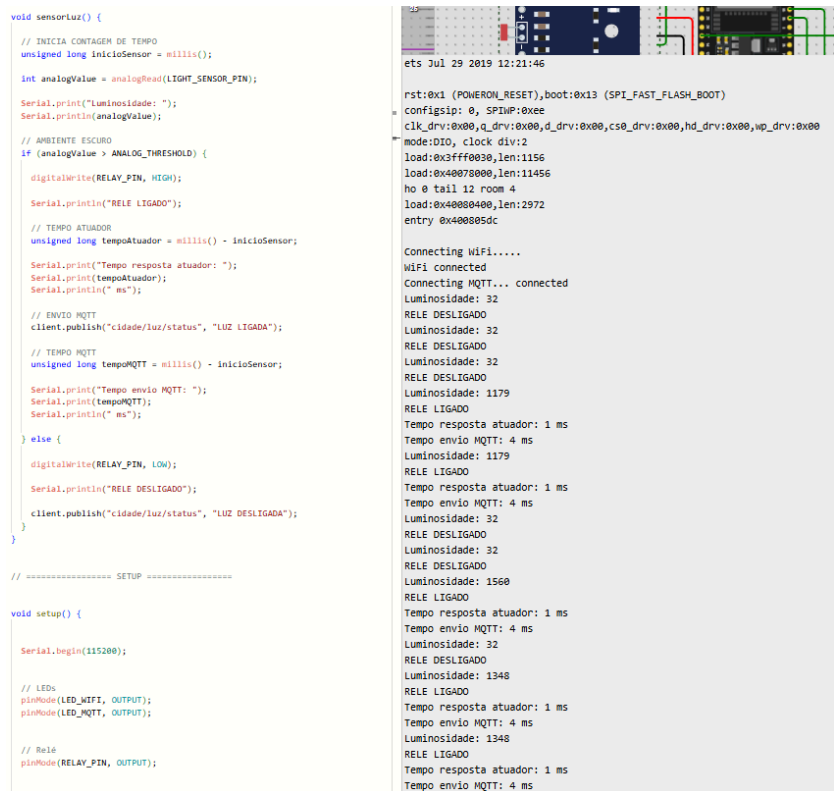


Figura 14 – Código/resultado de Análise de Desempenho

Tempo do atuador

Atuador LED	1 ms
Atuador LED	1 ms
Atuador LED	1 ms
Atuador LED	1 ms

Tempo MQTT

MQTT	4 ms
MQTT	4 ms
MQTT	4 ms
MQTT	4 ms

Média

Sensor LDR - Relé	1 ms
Sensor LDR - MQTT	4 ms

Os testes realizados demonstraram estabilidade no tempo de resposta do sistema. O acionamento do relé ocorreu em aproximadamente 1 ms após a detecção do sensor de luminosidade, enquanto o envio das mensagens MQTT apresentou média de 4 ms. Os resultados indicam baixa latência tanto no processamento local quanto na comunicação IoT, demonstrando eficiência na integração entre hardware, software e protocolo MQTT.

4. Repositório no Github

[GabrielMarc1/Implementa-o-IoT-de-luminosidade-com-ESP32](#)

5. Vídeo no Youtube

[Controle de Iluminação com ESP32](#)

6. Conclusão

O desenvolvimento do projeto permitiu a implementação de um sistema inteligente de iluminação urbana baseado em conceitos de Internet das Coisas (IoT), utilizando o microcontrolador ESP32, sensor de luminosidade LDR, módulo relé e comunicação via protocolo MQTT. Os objetivos propostos foram alcançados, uma vez que o sistema conseguiu realizar o monitoramento da luminosidade do ambiente, o acionamento automático da iluminação pública simulada e a comunicação via internet utilizando protocolos TCP/IP e MQTT.

Durante o desenvolvimento do projeto, alguns desafios foram encontrados, principalmente relacionados à configuração da comunicação Wi-Fi e integração do protocolo MQTT com o ESP32. Além disso, houve dificuldades na modelagem inicial do circuito eletrônico e na configuração correta das bibliotecas necessárias para comunicação MQTT no ambiente de simulação. Esses problemas foram solucionados por meio de ajustes no código, utilização de bibliotecas compatíveis com o ESP32 e testes realizados na plataforma Wokwi, permitindo a validação do funcionamento completo do sistema.

Entre as principais vantagens do projeto destacam-se o baixo custo de implementação, a automação do sistema de iluminação, a possibilidade de monitoramento em tempo real e a integração com infraestrutura IoT. Outro ponto relevante é a utilização de comunicação MQTT, protocolo amplamente empregado em aplicações de cidades inteligentes devido à sua eficiência e baixo consumo de recursos. Como desvantagens, podem ser citadas a dependência de conexão com a internet para funcionamento da comunicação remota e as limitações do ambiente de simulação em relação a testes realizados em cenários urbanos reais.

Como possibilidades de melhoria, o projeto pode futuramente incorporar sensores adicionais, integração com plataformas de monitoramento em nuvem, armazenamento de dados em banco de dados e dashboards para análise em tempo real do consumo energético da iluminação urbana. Além disso, seria possível implementar controle individual de múltiplos pontos de iluminação pública, tornando o sistema mais próximo de aplicações reais voltadas às cidades inteligentes e sustentáveis.

7. Referências

BRASIL. Objetivos de Desenvolvimento Sustentável – ODS 11: Cidades e Comunidades Sustentáveis. Plataforma ODS Brasil. Disponível em: <https://odsbrasil.gov.br/objetivo/objetivo?n=11>. Acesso em: 16 mar. 2026.

ROBOCORE. ESP32 WiFi Bluetooth. Disponível em: <https://www.robocore.net/wifi/esp32-wifi-bluetooth>. Acesso em: 01 abr. 2026.

MAKERHERO. O que é LDR? Sensor de luminosidade. Disponível em: <https://www.makerhero.com/blog/o-que-e-ldr/>. Acesso em: 01 abr. 2026.

ELETROGATE. LED alto brilho 5mm verde. Disponível em: <https://www.eletrogate.com/led-alto-brilho-5mm-verde>. Acesso em: 01 abr. 2026.

ELETROGATE. Resistor 1M 1/4W (10 unidades). Disponível em: <https://www.eletrogate.com/resistor-1m-1-4w-10-unidades>. Acesso em: 01 abr. 2026.

FRITZING. Fritzing: ferramenta de prototipagem eletrônica. Disponível em: <https://fritzing.org/>. Acesso em: 30 abr. 2026.

ESP32IO. ESP32 Light Sensor Tutorial. Disponível em: <https://esp32io.com/tutorials/esp32-light-sensor>. Acesso em: 30 abr. 2026.

ARDUINO. Arduino IDE. Disponível em: <https://www.arduino.cc/en/software>. Acesso em: 29 maio 2026.

WOKWI. ESP32 WiFi Guide. Disponível em: <https://docs.wokwi.com/guides/esp32-wifi>. Acesso em: 29 maio 2026.

HIVEMQ. HiveMQ MQTT Broker. Disponível em: <https://www.hivemq.com/>. Acesso em: 29 maio 2026.

MQTT. MQTT Official Website. Disponível em: <https://mqtt.org/>. Acesso em: 29 maio 2026.

KNOLLEARY. PubSubClient Library for Arduino. GitHub. Disponível em: <https://github.com/knolleary/pubsubclient>. Acesso em: 29 maio 2026.

KNOLLEARY. MQTT ESP8266 Example. GitHub. Disponível em: https://github.com/knolleary/pubsubclient/blob/master/examples/mqtt_esp8266/mqtt_esp8266.ino. Acesso em: 29 maio 2026.

WOKWI. Wokwi Simulator. Disponível em: <https://wokwi.com/>. Acesso em: 29 maio 2026.